

Personal Memory AI 한글 기획서

작성일: 2026-05-26

상태: 배포 우선 전환 후, 프론트 디자인 재정의 전 단계

작성 기준: Ko Yunseo 피드백, 현재 Railway staging, 하네스 audit 결과

1. 한 줄 정의

****Personal Memory AI는 내가 남긴 일기, 메모, Notion/Obsidian/Markdown 기록을 기억 구조로 바꾸고, 과거의 나를 근거로 질문·결정·패턴을 보여주는 개인 기억 AI다.****

핵심 문장:

나보다 나를 더 잘 아는 개인 기억 AI

그래프는 제품 그 자체가 아니라, AI가 왜 그렇게 말하는지 보여주는 ****증거 UI****다.

2. 현재 문제 인식

현재 staging 화면은 실제로 배포되고 렌더되지만, 디자인 관점에서는 아직 제품 화면이라고 보기 어렵다.

문제:

- Careerhacker Alex 같은 레퍼런스와 비교한 디자인 디테일 반영이 안 됨
- 정보 구조가 거칠고, 첫 화면의 제품적 설득력이 약함
- 그래프가 “멋진 시각화”보다는 데이터 덤프처럼 보일 수 있음
- UI hierarchy, typography, spacing, color, card depth, interaction affordance가 부족함
- 사용자가 처음 봤을 때 “내 기억을 이해하는 AI”라는 감정이 바로 오지 않음
- 기존 RPI 하네스가 프론트 디자인 품질을 검증하지 못했음

따라서 앞으로의 순서는 다음처럼 바꾼다.

1. Railway에 무중단 staging URL을 먼저 띄운다.
2. 실제 URL 기준으로 프론트 디자인을 먼저 수정한다.
3. 수정 결과는 반드시 이미지로 보고한다.
4. 디자인/브라우저 검증 후에만 RPI를 다시 돌린다.

3. 제품 표면 구분

3.1 App: 빠른 기억/일기 캡처

역할:

- 사용자가 하루 중 떠오르는 생각, 감정, 결정, 후회를 빠르게 저장
- 긴 글 작성보다 “빠른 기억 저장”이 목표
- 저장된 내용은 MemoryRecord로 정규화됨

초기 범위:

- 텍스트 중심 빠른 캡처
- 날짜/시간 자동 기록
- 감정, 프로젝트, 결정 힌트 선택 또는 자동 추출
- 추후 native app 또는 PWA로 확장

3.2 Web: second-brain 분석/그래프 workspace

역할:

- 캡처된 일기와 import된 메모를 그래프로 보여줌
- 기억 간 연결, 반복 패턴, 결정 결과, 감정 흐름을 설명
- Ask My Past Self, Decision Replay, Weekly Pattern Report 제공

초기 범위:

- 첫 화면은 memory brain graph
- 그래프 노드는 기억, 감정, 결정, 결과, 프로젝트, source
- 오른쪽 evidence drawer에서 모든 답변의 근거 확인
- Ask/Decision/Pattern은 그래프 위의 action panel로 제공

3.3 Backend: PostgreSQL + pgvector

역할:

- MemoryRecord 영구 저장
- embedding 저장
- semantic search

- graph evidence lookup
- user isolation
- export/delete

초기 staging에서는 Railway PostgreSQL + pgvector를 사용한다.

4. 핵심 사용자 흐름

Flow A: 빠른 기억 저장

1. 사용자가 app에서 짧은 기억을 입력한다.
2. 시스템이 MemoryRecord로 변환한다.
3. 감정, 결정, 프로젝트, outcome 후보가 붙는다.
4. web graph에 새로운 기억 node로 나타난다.

성공 기준:

- 저장 후 web에서 node가 보인다.
- node를 누르면 원문/요약/source/date가 보인다.

Flow B: 기존 기록 import

1. 사용자가 Notion/Obsidian/Markdown 기록을 가져온다.
2. 시스템이 중복, source, 날짜, 요약을 preview한다.
3. 사용자가 적용하면 MemoryRecord로 저장된다.
4. graph에 기존 기억들이 연결된다.

성공 기준:

- import preview에서 source/date/duplicate status가 보인다.
- 적용된 기록은 graph와 evidence drawer에 연결된다.

Flow C: Ask My Past Self

1. 사용자가 질문한다. 예: “이번에도 기능을 더 넣어야 할까?”
2. 시스템이 관련 기억을 검색한다.
3. citation이 충분하면 답변을 만든다.
4. 답변에는 citation, confidence, graph highlight가 포함된다.

5. 근거가 부족하면 일반 조언을 하지 않고 insufficient evidence를 표시한다.

성공 기준:

- 답변에는 반드시 citation이 있다.
- 그래프에서 관련 기억/감정/결정/outcome이 highlight된다.
- evidence drawer에서 근거를 확인할 수 있다.

Flow D: Decision Replay

1. 사용자가 현재 결정을 입력한다.
2. 과거 유사 결정과 결과를 찾는다.
3. 과거의 선택, 감정, 결과를 비교한다.
4. 현재 결정에 대한 recommendation과 uncertainty를 제시한다.

성공 기준:

- 유사 결정 목록이 citation과 함께 보인다.
- 추천은 과거 기억 근거에 묶여야 한다.
- generic advice 금지.

Flow E: Weekly Pattern Report

1. 일주일간 쌓인 기억을 분석한다.
2. 반복 감정, 결정 습관, 프로젝트 병목을 요약한다.
3. 근거 기억과 함께 report를 보여준다.

성공 기준:

- report는 citation 기반이다.
- insufficient evidence 상태가 명확히 표현된다.

5. 디자인 방향

5.1 현재 staging 디자인 평가

현재 화면은 “기능이 보이는 기술 데모”에 가깝다. 제품 화면으로 가려면 다음이 필요하다.

- 첫 화면에서 감정적 가치가 먼저 보여야 함

- 그래프는 중앙 증거 레이어지만 과도한 정보량을 줄여야 함
- Ask/Replay/Pattern은 카드형 action으로 더 명확히 배치
- evidence drawer는 “왜 이 답이 나왔는가”를 신뢰감 있게 설명해야 함
- status label은 내부 구현 상태가 아니라 사용자에게 필요한 범위만 노출

5.2 레퍼런스 비교 방향: Careerhacker Alex류 사이트

따라갈 디테일:

- 강한 headline과 짧은 value proposition
- 여백이 충분한 section 구성
- 하나의 핵심 demo를 먼저 보여주는 landing rhythm
- card depth, soft border, restrained palette
- 시선이 headline → graph/demo → CTA/action으로 이동하는 구조
- 지나친 데이터 나열보다 선별된 evidence story

반영할 화면 구조:

1. Hero: “나보다 나를 더 잘 아는 AI”
2. Main demo: Memory graph + Ask card
3. Evidence proof: cited memories drawer
4. Use cases: Ask / Decision Replay / Weekly Report
5. Capture/import: 앱 캡처와 import onboarding
6. Privacy/trust: 내 데이터, export/delete, citation only

6. 정보 구조

첫 화면

- 제품명
- 한 줄 가치
- primary action: 기억 추가 / Ask 시작
- memory brain graph
- 선택된 질문 또는 기억
- source
- date
- raw excerpt

- why connected
- Ask My Past Self
- Decision Replay
- Weekly Pattern Report
- Import Preview
- 상단 hero
- 중앙 demo
- 오른쪽 또는 하단 evidence panel
- 아래 action cards

디자인 원칙

- 그래프는 복잡하게 보이지 않게 progressive disclosure 적용
 - 처음에는 5~7개 핵심 node만 보여줌
 - 더 깊은 연결은 hover/click/drawer에서 표시
 - citation은 신뢰 요소로 반복 노출
 - skeleton/partial/internal status는 사용자 화면에서 과하게 드러내지 않음
-

7. 상태 라벨 정책

개발 내부 상태:

- implemented
- partial
- skeleton
- fake/sample
- planned
- blocked

사용자-facing 상태:

- 연결됨
- 근거 충분
- 근거 부족
- 준비 중
- 가져오기 필요

주의:

- 내부 라벨을 그대로 화면에 많이 노출하면 제품 완성도가 낮아 보인다.
 - 개발 리포트에는 내부 라벨을 유지하되, 제품 UI는 사용자 언어로 바꾼다.
-

8. 현재 staging 상태

현재 배포 URL:

- Web: <https://web-production-bcaf6.up.railway.app>
- API: <https://api-production-b11d.up.railway.app>

현재 확인:

- Web `/health/live`: OK
- API `/health/live`: OK
- API `/health/ready`: KMS/encryption 설정 때문에 not-ready
- Railway Postgres env presence: 확인됨, secret 값은 출력하지 않음

중요:

- 현재 web 화면은 최종 디자인이 아니다.
 - 현재 화면은 “배포가 되었고 기능 surface가 렌더된다”는 증거다.
 - 디자인 개선은 지금부터 별도 phase로 진행한다.
-

9. 하네스 문제와 수정 방향

기존 RPI 하네스 문제:

- Codex exit code와 좁은 TypeScript subset만 보고 verified 처리 가능
- frontend component 전체 검증 부족
- screenshot requirement가 prompt에만 있고 gate로 강제되지 않음
- PNG artifact가 실제 브라우저 screenshot인지 보장하지 않음
- review checklist가 markdown일 뿐 block하지 않음

수정된 운영 원칙:

1. RPI 자동 생성은 중지한다.

2. 배포 URL을 먼저 확보한다.
 3. `npm run typecheck`, `npm test`, `npm run build`를 통과한다.
 4. 실제 staging URL을 브라우저에서 열어 screenshot을 캡처한다.
 5. Ko Yunseo에게 실제 이미지를 첨부해서 보고한다.
 6. 그 후에만 RPI를 재개한다.
-

10. 앞으로의 개발 순서

Phase 1: 배포 안정화

완료:

- Railway web service 생성
- Dockerfile 기반 배포 성공
- public staging URL 확보
- health live 확인

남은 것:

- 도메인 이름 정리
- web/API 역할 문서화
- KMS readiness blocker 별도 처리

Phase 2: 프론트 디자인 재작업

작업:

- Careerhacker Alex 레퍼런스 분석
- 현재 staging UI 차이점 목록 작성
- hero/graph/evidence/action card layout 재설계
- typography, spacing, palette, card system 수정
- 내부 status label 과노출 제거
- 첫 화면 screenshot 보고

완료 기준:

- staging URL에서 실제 렌더 확인
- screenshot 첨부 보고

- 디자인 개선 전/후 차이 설명

Phase 3: 제품 기능 loop 강화

작업:

- Ask My Past Self interaction 강화
- Decision Replay interaction 강화
- import preview UX 강화
- fast diary capture UX 강화
- graph node click → evidence drawer 연결 강화

완료 기준:

- 각 flow별 실제 screenshot
- 브라우저 smoke 통과
- citation/evidence 유지

Phase 4: Postgres/pgvector 연결

작업:

- MemoryRecord persistence
- embeddings 저장
- semantic search
- graph evidence lookup
- per-user isolation
- export/delete smoke

완료 기준:

- secret 값 노출 없이 env presence 확인
- pgvector extension/vector smoke
- insert/search/delete 검증

Phase 5: RPI 재개

조건:

- staging URL 존재

- 프론트 디자인 1차 개선 완료
- 이미지 보고 완료
- 하네스 gate 강화 완료

RPI 재개 시:

- 한 cycle마다 배포/브라우저 검증 포함
- 프론트 작업은 이미지 없으면 complete 금지
- design acceptance를 별도 gate로 둔다

11. 완료 정의

MVP가 됐다고 말하려면 다음을 만족해야 한다.

- app capture 또는 capture prototype이 MemoryRecord를 만든다.
- Notion/Obsidian/Markdown import preview가 있다.
- web 첫 화면에서 memory graph가 제품적으로 설득력 있게 보인다.
- Ask My Past Self가 citation 기반 답변을 만든다.
- Decision Replay가 과거 결정 근거로 추천한다.
- evidence drawer에서 모든 AI action의 근거를 확인할 수 있다.
- staging URL에서 실제로 열린다.
- 주요 화면 screenshot이 보고된다.
- Postgres/pgvector staging plan 또는 smoke가 명확하다.
- skeleton/fake/sample이 완성처럼 보고되지 않는다.

12. 다음 즉시 작업

1. 현재 staging 디자인을 레퍼런스와 비교한다.
2. 첫 화면 정보 구조를 다시 잡는다.
3. UI를 제품 화면처럼 정리한다.
4. staging에 다시 배포한다.
5. 실제 screenshot을 첨부해서 보고한다.
6. 그 다음 RPI를 다시 돌린다.